Digital Design Lab

Ratko Pilipović

2026-10-23

Table of contents

Preface 1

I. Lab Exercises 3

1. First exercise: Tools and the Digital Design Cycle 5
1.1. The digital design cycle 5
1.2. Tools used in this course 6
1.3. Installing Anvil 8
1.4. Your first design: basic_comb 9
1.5. Starter files 13
1.6. Assignment 13

2. Second exercise: Introduction to (System)Verilog 15
2.1. Types 15
2.1.1. Logic (NetList) Types 15
2.1.2. Vector Types 15
2.1.3. Variable Types 15
2.1.4. Expressing Numbers Using Different Bases 16
2.2. Module 16
2.3. Operators 17
2.3.1. Numerical Operators 18
2.3.2. Logical Operators 18
2.3.3. Relational Operators 18
2.3.4. Bitwise Operators 19
2.3.5. Reduction Operators 19

2.3.6. Conditional Operator 20
2.3.7. Concatenation and Replication Operators 20
2.4. Modeling Concurent functionality in SystemVerilog 21
2.4.1. Continuous assignment 21
2.4.2. 4-bit 4:1 Multiplexer 22
2.4.3. 4-bit adder 23
2.4.4. User Constraints Files and Pin Planning in Vivado . 25
2.5. Assignment 26

3. Third exercise: Designing sequential circuits with SystemVerilog 29
3.1. Designing D Flip-Flop in SystemVerilog 29
3.2. Combinational logic with always statement 31
3.3. Common mistakes 33
3.4. Task: Implement a 4-bit Gray Code Counter 35
3.4.1. Example Code Structure: 37
3.5. Implementation guidelines 38
3.5.1. Divider 38
3.5.2. Up/down binary counter 40
3.5.3. Gray code converter 42
3.5.4. Pin mapping 43
3.6. Literature 44

Preface

Every processor, controller, and screen you have ever used began as a handful of logic gates someone arranged on purpose. This book is about learning to do that arranging yourself — not by reading about it, but by designing, simulating, and running your own digital circuits on real hardware.

Over nine labs, you will go from your first combinational circuit to a complete FPGA-based System on Chip: a HIPV, a Risc-V based processor, talking to peripherals you wrote, over a bus you configured, validated with a real oscilloscope. Along the way you will hit the same problems every digital designer hits — a signal that glitches, a counter that free-runs, a testbench that lies to you — and learn to reason your way out of them.

That debugging instinct, more than any single lab, is the point.

Note to Students: These notes are intended solely for educational purposes. They are not a replacement for attending lectures or consulting textbooks, but rather a complementary resource to support your learning.

Part I.

Lab Exercises

1. First exercise: Tools and the Digital Design Cycle

This first lab has two goals. First, to introduce the **digital design cycle** — the sequence of steps every digital circuit goes through, from an idea to a working bitstream on real hardware — so that you understand *why* each later lab does what it does. Second, to get your machine set up with the toolchain we use throughout the course, so the remaining labs are about designing hardware rather than fighting installers.

1.1. The digital design cycle

Every design in this course, however small, passes through the same stages:

1. **Specification** — describe, in plain language, what the circuit must do: its inputs, its outputs, and the relationship between them.
2. **RTL design entry** — write that behavior in a hardware description language. In this course, that language is SystemVerilog.
3. **Functional verification (simulation)** — before touching real hardware, simulate the design against a testbench to check it behaves as specified. Bugs are far cheaper to find here than on a board.
4. **Synthesis** — translate the RTL description into a gate-level netlist: a graph of primitive logic cells and the wires connecting them.

5. **Place & route (implementation)** — map that netlist onto the FPGA's actual resources (logic cells, routing wires, I/O pins), respecting the timing and pin constraints you provide.
6. **Bitstream generation** — package the placed-and-routed design into the binary configuration file the FPGA fabric loads on power-up.
7. **Programming** — load the bitstream onto the physical board.
8. **Hardware validation** — check that the programmed board actually does what you specified in step 1. If it doesn't, you go back to step 1 or 2 and iterate.

Steps 3–7 are exactly what a toolchain automates for you. Knowing this sequence matters because when something goes wrong, the first useful question is *which stage failed* — a syntax error is a design-entry problem, a mismatch with the testbench is a verification problem, an unroutable design is a place & route problem, and an LED that doesn't light despite a clean build is a hardware/pin-constraint problem. Each points you to a different fix.

1.2. Tools used in this course

In previous years this course used **Xilinx Vivado**, a proprietary, vendor-specific design suite. **Starting this year, we use Anvil** — a small, open-source CLI that drives an entirely open FPGA flow (no license, no account, no multi-GB installer). Anvil orchestrates the same stages described above using open tools:

Design cycle stage	Tool
Design entry	SystemVerilog (you write `top.sv`)
Simulation	Icarus Verilog (`iverilog`) + GTKWave, via `anvil test`

Design cycle stage	Tool
Synthesis	`sv2v` (SystemVerilog → Verilog) feeding **Yosys**
Place & route	**VPR** (part of the F4PGA project — Yosys + VPR together)
Bitstream generation	F4PGA
Programming	`openFPGALoader`

Anvil itself just wires these tools together behind a handful of commands (`anvil init`, `anvil build`, `anvil test`, `anvil program`) so you don't invoke each one by hand.

Anvil supports two FPGA boards, the **Digilent Nexys A7 100T** and the **Nexys A7 50T**. Both boards are used across the exercises in this course.

> **i** Nexys A7 50T
>
> Anvil supports both the **Nexys A7 100T** and the **Nexys A7 50T** (support for the 50T was added in Anvil 1.1.0).
> **Your own computer:** a normal installation works out of the box. If you installed Anvil before the 50T was added, run `anvil update` once.
> **Classroom computers:** these have an older Anvil and you cannot run `anvil update` there (it needs `sudo`). Apply the classroom patch instead; it needs no administrator rights and changes only files in your home directory:
>
> ```
> wget https://gist.githubusercontent.com/RatkoFri/d12d34610a58260a58a803eec74e7185/raw/anvi
> bash anvil-nexys50t.sh
> source ~/.bashrc
> ```
>
> Run it once per account; running it again does no harm. Check the

result with `anvil boards` (both boards should be listed) and `anvil doctor`. Then start a 50T project with:

```
anvil init --board Nexys-A7-50T --example switches-leds
```

1.3. Installing Anvil

Prerequisites: a native **Ubuntu 22.04 LTS** machine, or **WSL2** on Windows (the docs call out the handful of WSL-only steps, mainly around forwarding the board's USB connection, as "WSL only"). Docker support is not available yet.

The recommended path is the automated setup script:

```
curl -fsSL https://raw.githubusercontent.com/LogiSmith/toolchain-setup/main/i
```

If you'd rather review the script before running it (always a reasonable instinct when piping a script into a shell):

```
git clone https://github.com/LogiSmith/toolchain-setup.git
cd toolchain-setup
less install.sh
./install.sh
```

A successful run ends with a green `Toolchain installed and all tests passed` banner. Open a **new** shell afterward (so your updated `PATH`/aliases take effect) and verify with:

```
anvil doctor
```

For manual, tool-by-tool installation steps, WSL2-specific USB forwarding, and troubleshooting, see the full LogiSmith installation docs — that page is the canonical reference and is more likely to stay current than a copy pasted here.

> Editor tip: SystemVerilog syntax highlighting and linting in VS Code
>
> Plain `.sv` files have no syntax highlighting in VS Code out of the box. Install Verilog-HDL/SystemVerilog (publisher `mshr-h`) for SystemVerilog syntax highlighting and snippets, plus linting that reports syntax and elaboration errors directly in the editor as you type, instead of only after running `anvil build` or `iverilog`.
> The extension needs to be pointed at a linter. Since this course already installs **Icarus Verilog** as part of the Anvil toolchain, set `verilog.linting.linter` to `iverilog` in the extension's settings rather than installing a separate tool like Verilator or Verible just for this. The `iverilog` executable needs to be on your `PATH`, which the Anvil installer already takes care of.

1.4. Your first design: basic_comb

With `anvil doctor` passing, build a first design that exercises switches and LEDs directly, with no clock involved: a small combinational circuit.

```
mkdir basic_comb && cd basic_comb
anvil init --board Nexys-A7-100T
```

This scaffolds a project:

```
basic_comb/
```

```
config.json     # board + module configuration
top.sv          # top-level design - edit this
*.xdc           # pin constraints, copied from the board master
tb/             # testbenches
Makefile
```

1.4.0.1. What's in `config.json`

`config.json` is the per-project manifest every `anvil` command reads (`build`, `test`, `program`). `anvil init --board Nexys-A7-100T` fills it in for you, so you won't normally hand-edit it in this lab — but it's worth knowing what's there:

```
{
  "project": "basic_comb",
  "board": "Nexys-A7-100T",
  "top": "top",
  "target": "nexys4ddr",
  "partname": "xc7a100tcsg324-1",
  "device": "artix7",
  "ofl_board": "nexys_a7_100",
  "xdc": "Nexys-A7-100T-Master.xdc"
}
```

- **`project`** — the project's name, defaulting to the directory name.
- **`board`** — which entry of Anvil's board registry (`boards.json`) this project targets; everything below `top` is copied in from that registry at `anvil init` time.
- **`top`** — the top-level module name Anvil looks for when it builds your design. It defaults to `top`, which is exactly why the module in `top.sv` needs to be named `top` rather than `basic_comb`.
- **`partname`**, **`device`**, **`target`**, **`ofl_board`**, **`xdc`** — board-specific details (exact FPGA part, device family, Makefile build target,

`openFPGALoader` board identifier, and master constraints filename) that synthesis, place & route, and programming each need. You won't touch these directly for the Nexys A7 100T.

- **`modules`** (not shown above, since this design has none) — a list of reusable RTL module dependencies with pinned versions, e.g. `"uart@1.0.0"`, added automatically by `anvil addmodule`. You'll meet this once a later lab pulls in a shared peripheral block.

Replace the contents of `top.sv` with three switch inputs feeding four LED outputs through simple boolean logic. Anvil expects the top-level module in `top.sv` to be named `top`, so the logic below is the same as the basic_comb.sv starter file, just renamed from `basic_comb` to `top`:

```
// basic_comb - simple 3-input combinational logic
module top (
    input  logic a,        // SW0
    input  logic b,        // SW1
    input  logic c,        // SW2
    output logic [3:0] y   // LED0-LED3
);
    assign y[0] = a & b | c;
    assign y[1] = a & c;
    assign y[2] = a | c;
    assign y[3] = a & b & c;
endmodule
```

1.4.0.2. Explanation

- **Continuous assignment**: each `assign` statement drives its output from a boolean expression of the inputs, re-evaluated any time `a`, `b`, or `c` changes. There is no clock and no memory — the outputs are a pure function of the current switch positions, which is exactly what "combinational logic" means.

- **This is the design-entry stage**: `assign` is the idiomatic way to describe combinational logic in SystemVerilog, the same way `always_ff` (which you'll meet in the next lab) is the idiom for sequential logic.
- **Four independent outputs**: `y[0]`–`y[3]` are four separate circuits sharing the same three inputs — nothing requires a module to have only one output bit.

Uncomment (or check) the pins for `a`, `b`, `c`, and `y` in the generated `.xdc` file — they should look like this:

```
set_property PACKAGE_PIN J15 [get_ports {a}]
set_property IOSTANDARD LVCMOS33 [get_ports {a}]

set_property PACKAGE_PIN L16 [get_ports {b}]
set_property IOSTANDARD LVCMOS33 [get_ports {b}]

set_property PACKAGE_PIN M13 [get_ports {c}]
set_property IOSTANDARD LVCMOS33 [get_ports {c}]

set_property PACKAGE_PIN H17 [get_ports {y[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {y[0]}]

set_property PACKAGE_PIN K15 [get_ports {y[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {y[1]}]

set_property PACKAGE_PIN J13 [get_ports {y[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {y[2]}]

set_property PACKAGE_PIN N14 [get_ports {y[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {y[3]}]
```

This maps `a`, `b`, `c` to switches SW0–SW2 and `y[0]`–`y[3]` to LED0–LED3 on the Nexys A7 100T. Unlike Blinky, there's no `create_clock` line here

— a purely combinational design has no clock port for the timing analyzer to check.

Then build and program the board:

```
anvil build       # sv2v → Yosys → VPR → bitstream, a few seconds
anvil program     # flash the bitstream to the board via openFPGALoader
```

If everything worked, flipping SW0–SW2 should change the LED0–LED3 pattern according to the four equations above.

1.5. Starter files

- basic_comb.sv — the same design used in the walkthrough above, as a ready-to-copy file (module named `basic_comb`; rename to `top` when you copy it into `top.sv`, as noted above).

1.6. Assignment

1. Run `anvil doctor` and confirm every check passes. If something fails, consult the troubleshooting guide before asking for help.
2. Build and program the `basic_comb` design above, and confirm the LED pattern matches the equations for every switch combination.
3. Extend the design with a fifth output, `y[4] = a ^ b ^ c` (XOR of all three switches), wired to LED4. Rebuild, reprogram, and confirm the new output on hardware.

2. Second exercise: Introduction to (System)Verilog

2.1. Types

Commonly used types in SystemVerilog include:

2.1.1. Logic (NetList) Types

- `logic`: A 4-state data type (0, 1, X, Z) used for modeling combinational and sequential logic.
- `bit`: A 2-state data type (0, 1) used for modeling binary data.

2.1.2. Vector Types

- `bit [n-1:0]`: An n-bit unsigned integer.
- `logic [n-1:0]`: An n-bit unsigned integer with 4-state logic.

2.1.3. Variable Types

- `logic`: A 4-state data type (0, 1, X, Z) used for modeling combinational and sequential logic.
- `byte`: An 8-bit signed integer.

- `shortint`: A 16-bit signed integer.
- `int`: A 32-bit signed integer.
- `longint`: A 64-bit signed integer.
- `integer`: A general-purpose 32-bit signed integer (legacy from Verilog).
- `real`: A 64-bit floating point variable.

2.1.4. Expressing Numbers Using Different Bases

```
// Binary representation (base 2)
logic [3:0] binary_num = 4'b1010; // 4-bit binary number 1010 (decimal 10)

// Octal representation (base 8)
logic [5:0] octal_num = 6'o12; // 6-bit octal number 12 (decimal 10)

// Decimal representation (base 10)
logic [7:0] decimal_num = 8'd10; // 8-bit decimal number 10

// Hexadecimal representation (base 16)
logic [7:0] hex_num = 8'hA; // 8-bit hexadecimal number A (decimal 10)
```

2.2. Module

All system modules in Verilog are encapsulated inside a module. Modules can include instantiation of lower-level modules to support hierarchical designs. The keywords `module` and `endmodule` mark the beginning and the end of the system description.

```
module ModuleName(
    portType signalType Port1_name,
    portType signalType Port2_name, Port3_name,
    .......
    portType signalType PortN_name
);
```

where the `portType` specifies the direction of the port (input, output, or inout), and `signalType` defines the type of the signal, as described in the previous section.

Port Type	Description
`input`	Specifies an input port to the module. Data flows into the module through this port.
`output`	Specifies an output port from the module. Data flows out of the module through this port.
`inout`	Specifies a bidirectional port. Data can flow both into and out of the module through this port.

2.3. Operators

SystemVerilog supports a variety of operators that are used for different purposes, including arithmetic, logical, relational, and bitwise operations. Here are some common operators with examples:

2.3.1. Numerical Operators

- `+` : Addition
- `-` : Subtraction
- `*` : Multiplication
- `/` : Division
- `%` : Modulus

```
logic [5:0] c;

// in1 and in2 inputs of module
assign c = in1 + in2;
```

2.3.2. Logical Operators

- `&&` : Logical AND
- `||` : Logical OR
- `!` : Logical NOT

```
logic c;

// in1 and in2 inputs of module
assign c = in1 && in2;
```

2.3.3. Relational Operators

- `==` : Equality
- `!=` : Inequality
- `<` : Less than
- `<=` : Less than or equal to
- `>` : Greater than

- `>=` : Greater than or equal to

```
logic c;

// in1 and in2 inputs of module
assign c
assign c = in1 >= in2;
```

2.3.4. Bitwise Operators

- `&` : Bitwise AND
- `|` : Bitwise OR
- `^` : Bitwise XOR
- `~` : Bitwise NOT
- `<<` : Logical shift left
- `>>` : Logical shift right
- `<<<` : Arithmetic shift left
- `>>>` : Arithmetic shift right

```
logic [5:0] c;

// in1 and in2 inputs of module
assign c = in1 & in2;
```

2.3.5. Reduction Operators

- `&` : Reduction AND
- `|` : Reduction OR
- `^` : Reduction XOR
- `~&` : Reduction NAND
- `~|` : Reduction NOR

- `~^` or `^~` : Reduction XNOR

```
logic [5:0] a;
logic c,d;
assign a = 6'b101010;

// in1 and in2 inputs of module
assign c = &a; // AND reduction result 0
assign d = |a; // OR reduction result 1
```

2.3.6. Conditional Operator

- `? :` : Conditional (ternary) operator

```
logic [5:0] a,b;
logic c;

assign c = (a > b) ? 1'b1 : 1'b0; // cc = either one or zero
```

2.3.7. Concatenation and Replication Operators

The concatenation operator `{}` in SystemVerilog is used to join multiple bits, vectors, or expressions into a single vector.

```
logic [3:0] a = 4'b1010;
logic [3:0] b = 4'b0101;
logic [7:0] c;

// Concatenate a and b to form an 8-bit vector
assign c = {a, b}; // c = 8'b10100101
```

You can also use the replication operator `{n{}}` to replicate a bit or vector `n` times.

```
logic [1:0] d = 2'b11;
logic [5:0] e;

// Replicate d three times to form a 6-bit vector
assign e = {3{d}}; // e = 6'b111111
```

2.4. Modeling Concurent functionality in SystemVerilog

2.4.1. Continuous assignment

Continuous assignment in SystemVerilog allows for the assignment of values to nets using the `assign` keyword, ensuring that the assigned value is continuously updated whenever the right-hand side expression changes. A continuous assignment models combinational logic. For example:

```
assign out = a & b; // 'out' is continuously assigned the result of 'a & b'
```

We need to differ continous assignment from assignment in programming languages.

First example:

```
assign b = a;
assign c = b;
```

In programming languages, a similar construct will first assign the value of `a` to `b`, and then pass the value of `b` to `c`. During synthesis, the tool will remove signal `b` and infer `assign c = a`.

Second example:

```
assign c = a;
assign c = b;
```

In programming languages, a similar construct will first assign the value of `a` to `c`, and then assign the value of `b` to `c`. The final value of `c` will be determined by the last assignment. During synthesis, the tool will create a net driven by two signals. As these signals by have different values, the net `c` will have an unknown value `X`. This situation must be avoided at all costs.

2.4.2. 4-bit 4:1 Multiplexer

The 4-bit multiplexer (MUX) selects one of the four 4-bit inputs to pass to the output based on the 2-bit selection signal.

```
module mux4to1 (
    input logic [3:0] in0, // 4-bit input 0
    input logic [3:0] in1, // 4-bit input 1
    input logic [3:0] in2, // 4-bit input 2
    input logic [3:0] in3, // 4-bit input 3
    input logic [1:0] sel, // 2-bit selection signal
    output logic [3:0] out // 4-bit output
);
    // Nested conditional operator
    // C = (A > B) ? 1 : (A < B) ? 0 : 1
    assign out = (sel == 2'b00) ? in0 :
                 (sel == 2'b01) ? in1 :
```

```
                    (sel == 2'b10) ? in2 :
                    (sel == 2'b11) ? in3 :
                    4'b0000; // Default case

endmodule
```

2.4.2.1. Explanation

- **Module Declaration**: The module `mux4to1` is declared with inputs and outputs.
- **Inputs**: Four 4-bit inputs (`in0`, `in1`, `in2`, `in3`) and a 2-bit selection signal (`sel`).
- **Output**: One 4-bit output (`out`).
- **Conditional Statement**: The `assign` statement uses a series of conditional (ternary) operators to select one of the four inputs based on the value of the 2-bit selection signal (`sel`).

2.4.3. 4-bit adder

The 4-bit adder adds two 4-bit binary numbers and produces a 4-bit sum and a carry-out bit.

```
module adder4bit (
    input logic [3:0] a, // 4-bit input a
    input logic [3:0] b, // 4-bit input b
    output logic [3:0] sum, // 4-bit sum output
    output logic carry_out // Carry-out bit
);

    logic c1, c2, c3; // Intermediate carry bits
```

```
    // Half Adder for the least significant bit
    assign sum[0] = a[0] ^ b[0];
    assign c1 = a[0] & b[0];

    // Full Adder for the second bit
    assign sum[1] = a[1] ^ b[1] ^ c1;
    assign c2 = (a[1] & b[1]) | (c1 & (a[1] ^ b[1]));

    // Full Adder for the third bit
    assign sum[2] = a[2] ^ b[2] ^ c2;
    assign c3 = (a[2] & b[2]) | (c2 & (a[2] ^ b[2]));

    // Full Adder for the most significant bit
    assign sum[3] = a[3] ^ b[3] ^ c3;
    assign carry_out = (a[3] & b[3]) | (c3 & (a[3] ^ b[3]));

endmodule
```

2.4.3.1. Explanation

- **Module Declaration**: The module `adder4bit` is declared with two 4-bit inputs (`a` and `b`), a 4-bit output (`sum`), and a carry-out bit (`carry_out`).
- **Intermediate Carry Bits**: Three intermediate carry bits (`c1`, `c2`, `c3`) are used to store the carry from each adder stage.
- **Half Adder**: The least significant bit is added using a half adder, which produces the sum and the first carry bit (`c1`).
- **Full Adders**: The next three bits are added using full adders, which take into account the carry from the previous stage.
- **Final Carry-Out**: The carry-out from the most significant bit is assigned to `carry_out`.

2.4.4. User Constraints Files and Pin Planning in Vivado

In Vivado, User Constraints Files (UCF) or Xilinx Design Constraints (XDC) files are used to define the physical constraints of the design, such as pin assignments, timing constraints, and other implementation-specific parameters. Pin planning involves mapping the logical signals in your design to the physical pins on the FPGA. This process ensures that the design interfaces correctly with external components and meets the required performance criteria. Proper pin planning and constraint management are crucial for successful FPGA implementation, as they directly impact signal integrity, timing closure, and overall design functionality.

The XDC file for Nexys A7 defines the pin placement for external signals of modules. To map the inputs of our `adder4bit` to the switches and the outputs to LEDs, we need to make the following changes:

```
##Switches
set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { a[0] }]; #IO_L24N_
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { a[1] }]; #IO_L3N_T
set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { a[2] }]; #IO_L6N_T
set_property -dict { PACKAGE_PIN R15   IOSTANDARD LVCMOS33 } [get_ports { a[3] }]; #IO_L13N_
set_property -dict { PACKAGE_PIN R17   IOSTANDARD LVCMOS33 } [get_ports { b[0] }]; #IO_L12N_
set_property -dict { PACKAGE_PIN T18   IOSTANDARD LVCMOS33 } [get_ports { b[1] }]; #IO_L7N_T
set_property -dict { PACKAGE_PIN U18   IOSTANDARD LVCMOS33 } [get_ports { b[2] }]; #IO_L17N_
set_property -dict { PACKAGE_PIN R13   IOSTANDARD LVCMOS33 } [get_ports { b[3] }]; #IO_L5N_T

##LEDS
## LEDs
set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { sum[0] }]; #IO_L18
set_property -dict { PACKAGE_PIN K15   IOSTANDARD LVCMOS33 } [get_ports { sum[1] }]; #IO_L24
set_property -dict { PACKAGE_PIN J13   IOSTANDARD LVCMOS33 } [get_ports { sum[2] }]; #IO_L17
set_property -dict { PACKAGE_PIN N14   IOSTANDARD LVCMOS33 } [get_ports { sum[3] }]; #IO_L8P
set_property -dict { PACKAGE_PIN R18   IOSTANDARD LVCMOS33 } [get_ports { carry_out }]; #IO_
```

It is important to assign the appropriate pin to the `get_ports` function.

2.5. Assignment

Your task is to implement a "dummy" 4-bit Arithmetic Logic Unit (ALU) with the following interface: - **a, b**: 4-bit inputs - **alu_control**: 3-bit input for selecting the operation - **c**: 4-bit output

`alu_control` Value	Operation
000	Addition
001	Subtraction
010	Bitwise AND
011	Bitwise OR
100	Bitwise XOR
101	Compare (equal)
110	Pass-through `b`
111	Pass-through `a`

> **Note**
>
> The ALU is 4-bit rather than 6-bit specifically to keep addition/subtraction small. The open-source Artix-7 flow (F4PGA: Yosys + VPR) that Anvil uses has known weak spots mapping inferred carry-chain arithmetic (`+`/`-`) — the same `CARRY4` logic behind the `adder4bit` example above — and wider inferred adders can blow up synthesis time and resource usage. Staying at 4 bits, matching `adder4bit`'s width, keeps you well clear of that.

Tips: 1. Employ operators and intermediate signals to obtain the results of each operation. 2. Connect input signals to the rightmost switches (SW0 to SW7), and selection signals to the leftmost switches (SW13 to

SW15). The output signal should be connected to LEDs (LED0 to LED3).
3. Employ nested conditionals.

3. Third exercise: Designing sequential circuits with SystemVerilog

Vivado recognizes certain idioms and transforms them into specific sequential circuits. Other coding styles may simulate correctly but may synthesize into circuits with obvious or subtle errors. We will present the proper idioms to describe registers and latches.

3.1. Designing D Flip-Flop in SystemVerilog

The Verilog code for a D flip-flop (DFF) active on the positive clock edge with a synchronous reset is given below:

```
module DFF (
    input logic clk,
    input logic reset,
    input logic d,
    output logic q
);
    always_ff @(posedge clk) begin
        if (reset)
            q <= 1'b0;
        else
            q <= d;
```

```
    end
endmodule
```

The `always_ff` has it roots from Verilog language. In Verilog we have `always` statement, written in form:

```
always @(sensitivity_list)
    statement;
```

In the previous code, the statement is executed only when the event specified in the sensitivity list occurs. For the D-FF example, the event is `posedge clk`, meaning the statements inside `always_ff` will execute on the positive edge of the clock. Inside the `always_ff` block, we use an `if/else` statement for conditional operations. Notice that the non-blocking operator `<=` is used inside the `always_ff` block. On every positive edge of the clock, the block checks if the reset is active. If the reset is active, the output will be set to zero; otherwise, the input will be copied to the output. The `always_ff` construct behaves like the `always` construct but is specifically used to imply flip-flops and allows tools to produce a warning if anything else is implied.

Let us design 4-bit counter, which counts on positive edge and has active high reset

```
module Counter4Bit (
    input logic clk,
    input logic reset,
    output logic [3:0] out
);
    logic [3:0] counter;

    always_ff @(posedge clk) begin
        if (reset)
```

```
            counter <= 4'b0000;
        else
            counter <= counter + 1;
    end

    assign out = counter;
endmodule
```

As in the previous example, every time `posedge clk` occurs, the counter will increment the value of the internal counter `counter` if `reset` is low. Otherwise, the counter will start counting from zero. Finally, we output the state of the internal counter to the output. Note that `assign` and `always_ff` work in parallel.

3.2. Combinational logic with always statement

We can design the combinational logic using `always` constructs. Let us first describe an inverter using `always` constructs.

```
module Inverter (
    input logic a,
    output logic y
);
    always_comb begin
        y = ~a;
    end
endmodule
```

In this example, the `always_comb` block is used to describe combinational logic. The output `y` is always the logical NOT of the input `a`. The

`always_comb` construct ensures that the block is always executed whenever any of its inputs change, making it suitable for combinational logic. In the inverter code we use the blocking operator `=`.

> **Note:**
>
> In SystemVerilog, a group blocking assignments (`=`) are evaluated in order they appear. They are used for combinational logic. Non-blocking assignments (`<=`) are used in sequential logic. A group of non-blocking statements is evaluated concurently.

Let us construct 4-to-1 MUX using `always_comb` and `case` construct

```
module Mux4to1 (
    input logic [1:0] sel,
    input logic [3:0] d,
    output logic y
);
    always_comb begin
        case (sel)
            2'b00: y = d[0];
            2'b01: y = d[1];
            2'b10: y = d[2];
            2'b11: y = d[3];
            default: y = 1'b0;
        endcase
    end
endmodule
```

In this example, the `always_comb` block is used to describe the combinational logic for a 4-to-1 multiplexer. The `case` statement selects one of the four input bits `d` based on the 2-bit selector `sel` and assigns it to the

output `y`. The `default` case ensures that `y` is set to `0` if `sel` has an unexpected value. Be sure to always give `default` option in the `case` construct, otherwise you will infer a latch in your design.

3.3. Common mistakes

1. There is no assign construct in always block:

```
module WrongModule1 (
    input logic a,
    input logic b,
    output logic c
);
    always_comb begin
        assign c = a & b;
    end
endmodule
```

2. Having multiple blocks driving the same net

```
module WrongModule2
    ....
    always_ff(@posedge clock) begin
        ......
        c <= {x,1'b0};
    end
    ....
    always_comb begin
        ......
        c = x;
    end
    ....
```

```
    assign c = in1 + in2;
endmodule
```

3. Infering latch in combinational design:

```
logic result;
logic status;

always_comb begin
    if (input_button) begin
        status = 1'b1;
        result = 1'b0;
    end
    else if (done_signal) begin
            status = 1'b0;
        end
    end
```

What is the value of `result` when `done_signal` is high? What will happen when `input_button` and `done_signal` are low? The hardware synthesizer will assume the old value, thereby inferring a latch. You do not want a latch in a combinational design.

Correct answer would be:

```
logic result;
logic status;

always_comb begin
    if (input_button) begin
        status = 1'b1;
        result = 1'b0;
    end
    else if (done_signal) begin
```

```
            status = 1'b1;
            result = 1'b0;
        end
        else begin
            status = 1'b0;
            result = 1'b0;
        end
    end
```

Remember cover all your cases!

3.4. Task: Implement a 4-bit Gray Code Counter

Your task is to implement a 4-bit Gray Code counter, that changes its state with the frequency of 1 Hz.

Steps required: 1. Start with two counters: a modulo 100*10^6 counter and a classical binary counter that counts from 0 to 15. 2. The first counter increments its value every system clock cycle. 3. The other counter changes its value when the first counter reaches the value 100*10^6-1. When the input *up_down* is equal to 0, the counter increments its value; otherwise it decreases. 4. Convert the state of the binary counter to Gray Code.

Python code snippet that illustrates the behaviour of gray code counter

```
binary_count = 0;

while(1):
    for i in range (0, 100 * 1e6): # the module 100*1e6 counter
        if(i==100 * 1e6-1):
            if(up_down==0):
                binary_count = binary_count + 1
```

```
            else:
                binary_count = binary_count - 1

    gray_code = bin2gray(binary_count )
```

```
def bin2gray(binary):
    match binary:
        case 0: return 0
        case 1: return 1
        case 2: return 3
        case 3: return 2
        case 4: return 6
        case 5: return 7
        case 6: return 5
        case 7: return 4
        case 8: return 12
        case 9: return 13
        case 10: return 15
        case 11: return 14
        case 12: return 10
        case 13: return 11
        case 14: return 9
        case 15: return 8
        case _: return 0
```

3.4.1. Example Code Structure:

> **Name the top module `top`**
>
> Anvil looks for a top-level module called `top` in `top.sv` when it builds your design (see the `top` entry in the project configuration in the first exercise). Your design is the top-level module, so name it `top` and put it in `top.sv`. A different module name, such as `gray_code_counter`, would not be found as the top-level module.

```
module top (
    input logic clk,
    input logic rst, // Button
    input logic up_down, // Switch for up/down counting
    output logic [3:0] gray_code // connect the output to LEDs
);

    logic en;
    logic [3:0] binary_count;

    // Prescaler code & binary counter


    // Gray Code Converter
    // employ always_comb and case statement
    // to convert from code to another

endmodule
```

Tip: Binary to graycode converter

3.5. Implementation guidelines

Structural modeling, splitting a design into separate modules and instantiating them, is introduced in the next exercise. For now `top` stays a single module, so the three pieces below are not separate modules with their own ports, but three cooperating pieces of behavioral code inside `top`, sharing the internal signals `en` and `binary_count` already declared in the skeleton above. These three pieces of code are actually three separate `always` blocks, where first two are `always_ff` and the last one is `always_comb`. The first two blocks are sequential, the last one is combinational. The three pieces are described below.

3.5.1. Divider

The divider turns the 100 MHz system clock into a single, once-per-second pulse on `en`. It is a free running counter, the same idea you will meet again as the standalone `prescaler` module in the next exercise.

Declare an internal register wide enough to count up to 100,000,000:

```
logic [26:0] div_count; // 2^27 > 100,000,000
```

Behavior:

- On every rising edge of `clk`, if `rst` is high, reset `div_count` to zero.
- Otherwise, increment `div_count` by one on every rising edge. This is step 2 from above, the first counter incrementing every system clock cycle.
- When `div_count` reaches 100,000,000 - 1, drive `en` high for that clock cycle and let `div_count` wrap back to zero on the next edge, restarting the count.
- The result is a single clock cycle pulse on `en` once every 100,000,000 clock cycles. At 100 MHz that is exactly once per second.

The waveform below shows this behavior. Counting to 100,000,000 would be impossible to draw, so the diagram uses a much smaller modulus, $N = 6$. The idea is identical, only the number of clock cycles per pulse differs.

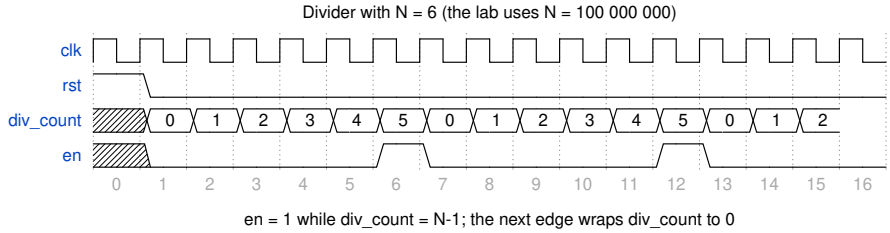


Figure 3.1.: Divider timing for $N = 6$. `div_count` counts $0, 1, \ldots, N-1$ and wraps. `en` is high for exactly one clock cycle, the one in which `div_count` equals $N - 1$.

Things to notice in Figure 3.1:

- While `rst` is high, `div_count` is cleared on the next rising edge. Before that first edge its value is unknown (hatched area).
- `div_count` changes only right after a rising edge of `clk`. Between edges it holds its value.
- `en` is high during the whole clock cycle in which `div_count` is $N-1$ (5 in the diagram, 99,999,999 in the lab). On the following edge `div_count` wraps to 0 and `en` falls again.
- The distance between two `en` pulses is exactly N clock cycles.

The reset is synchronous, so it takes effect on a clock edge, not immediately. If `rst` is pressed in the middle of a count, `div_count` restarts from zero and the pulse is delayed accordingly:

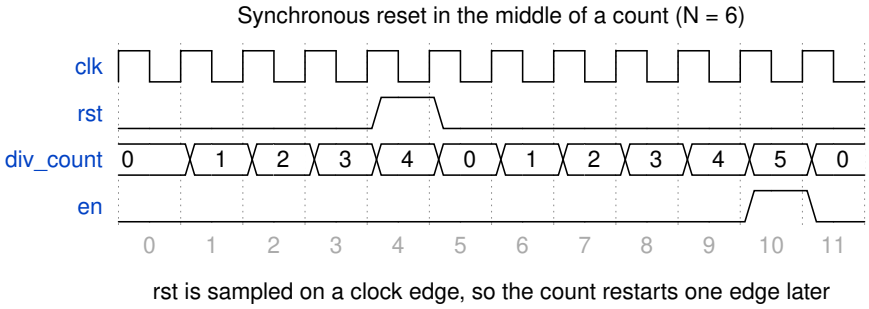


Figure 3.2.: Synchronous reset during counting for $N = 6$. The count restarts from 0 and the next `en` pulse appears N cycles after the restart.

3.5.2. Up/down binary counter

`binary_count` is the 4-bit counter whose value eventually gets Gray encoded for the LEDs. Unlike `div_count`, it does not advance on every clock edge. It only advances once per second, on a clock edge where `en` is high.

Behavior:

- On every rising edge of `clk`, if `rst` is high, reset `binary_count` to zero.
- Otherwise, only change `binary_count` on a clock edge where `en` is high. When `up_down` is 0, increment `binary_count`, wrapping from 15 back to 0. When `up_down` is 1, decrement it, wrapping from 0 back to 15.
- On a clock edge where `en` is low, hold `binary_count` at its current value.
- Because `en` pulses only once per second, `binary_count` also only changes once per second, which is where the counter's 1 Hz update

rate comes from.

The waveform below shows the counter counting up. The `en` pulses are drawn only 4 clock cycles apart so that several counter steps fit on the page.

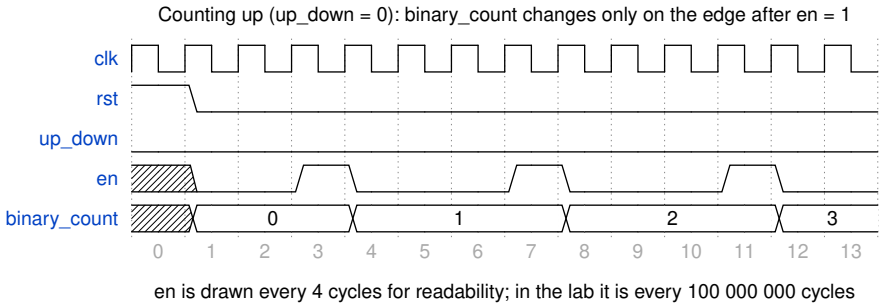


Figure 3.3.: Up counting (`up_down` = 0). `binary_count` is cleared by `rst`, then increments by one on the clock edge that follows each `en` pulse and holds its value in between.

Things to notice in Figure 3.3:

- `binary_count` never changes in the same cycle in which `en` is high. It changes on the *next* rising edge, because the flip-flops sample `en` at the edge. This is the same behavior as the `d` and `q` of the D flip-flop earlier in this chapter.
- Between `en` pulses the counter holds its value.

With `up_down` = 1 the counter decrements instead, and wraps from 0 to 15:

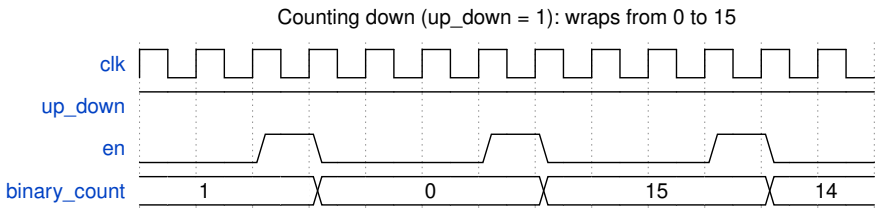


Figure 3.4.: Down counting (`up_down` = 1). The counter wraps from 0 to 15 because `binary_count` is only 4 bits wide.

3.5.3. Gray code converter

The converter has no state of its own. It is a purely combinational function of `binary_count`, mapping each of the 16 possible values to the corresponding Gray code word from the `bin2gray` table above.

Behavior:

- Use `always_comb` with a `case` statement on `binary_count`, one branch per value 0 through 15, assigning the matching Gray code word to `gray_code` in each branch.
- Always include a `default` branch, even though cases 0 through 15 already cover every possible 4-bit value. The Common mistakes section earlier in this chapter explains why an incomplete `case` without a `default` infers a latch instead of the combinational logic you intended.
- Because the conversion is combinational, `gray_code` updates immediately whenever `binary_count` changes. No separate clocking is needed here. `binary_count` already only changes once per second, so `gray_code` will too.

Table 3.1 lists the mapping that your `case` statement has to implement. It is the same mapping as the `bin2gray` function above.

Table 3.1.: Binary to Gray code conversion.

Decimal	`binary_count`	`gray_code`	`gray_code` (decimal)
0	0000	0000	0
1	0001	0001	1
2	0010	0011	3
3	0011	0010	2
4	0100	0110	6
5	0101	0111	7
6	0110	0101	5
7	0111	0100	4
8	1000	1100	12
9	1001	1101	13
10	1010	1111	15
11	1011	1110	14
12	1100	1010	10
13	1101	1011	11
14	1110	1001	9
15	1111	1000	8

Notice that between any two consecutive rows, including the wrap from 15 back to 0, exactly one bit of `gray_code` changes. This is the property that makes Gray code useful. The table follows a simple rule: the most significant bit is copied, and every other Gray bit is the XOR of the corresponding binary bit and the binary bit above it, $g_3 = b_3$ and $g_i = b_{i+1} \oplus b_i$ for $i = 2, 1, 0$. You may use the table directly in the `case` statement. The rule is a useful way to check your entries.

3.5.4. Pin mapping

Connect the `clk` signal to the system clock (100 MHz), the `rst` signal to the middle button, the `up_down` signal to the switch, and the `gray_code`

signal to the LEDs.

```
## Clock signal
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { cl
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports


##Switches
set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { up

## LEDs
set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { gra
set_property -dict { PACKAGE_PIN K15   IOSTANDARD LVCMOS33 } [get_ports { gra
set_property -dict { PACKAGE_PIN J13   IOSTANDARD LVCMOS33 } [get_ports { gra
set_property -dict { PACKAGE_PIN N14   IOSTANDARD LVCMOS33 } [get_ports { gra

##Buttons
set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports { rst
```

3.6. Literature

1. UC Davis Notes, Verilog: Common mistakes